记账簿 · 从零到独立开发

一套以真实项目为教材的网课讲义

这套讲义是什么

你手上有一个能跑的软件（记账簿），但你不知道它是怎么工作的。 这套讲义要做的事只有一件：**让你从"能指挥它"走到"能自己造它"。**

它和市面上的编程课有三点不同：

1. **教材就是你自己的项目。** 不讲"学生管理系统""图书管理"，从头到尾只讲记账簿——每个概念都对应你文件夹里一个真实文件。
2. **先建立直觉，再上术语。** 先讲"后厨和服务员不能直接说话，所以要有传菜单"，再告诉你这叫 IPC。反过来讲，你会被术语挡住。
3. **每讲都有动手。** 看懂和会做之间隔着一条河，只有敲键盘能过去。

你的目标与路线

你的最终目的是 **C：真懂原理，能独立做别的软件。** 但直接从原理讲起，第三讲就会放弃。所以路线是**螺旋上升**：

```
模块 0  热身              → 把环境跑起来，别怕它
模块 1  程序是什么        → 建立"计算机在干什么"的基本图景
模块 2  项目骨架          → 认识记账簿的每个零件
模块 3  会读代码          → 能顺着一条数据流读完，不迷路
模块 4  会改代码          → 能加一个功能、修一个 bug
模块 5  质量与安全        → 明白那些 verify 脚本在防什么
模块 6  打包发布          → 能把软件交到别人手上
模块 7  独立造东西        → 离开记账簿，从零做一个小软件
```

关键设计：每个模块都同时包含"懂原理"和"能动手"，但比例在变。

模块	讲原理	动手	你在这一段的身份
0–2	70%	30%	参观者：看懂就行
3–4	40%	60%	学徒：跟着做
5–6	30%	70%	操作员：独立完成任务
7	20%	80%	建造者：自己设计

到模块 7 时，你已经在不自觉中把 C 需要的东西学完了——因为你**先会做，再回头理解为什么。**

课时安排（我替你定的）

每讲 40 分钟，每周 2 讲，全套 24 讲 ≈ 12 周。

为什么是 40 分钟：

- **少于 25 分钟**：环境刚打开、脑子刚热，就结束了，下次要重新热身，效率极低。
- **多于 45 分钟**：零基础的新概念堆叠会开始互相干扰，你以为懂了，其实只是记住了字面。
- **40 分钟刚好装得下"一个完整的小胜利"**：读完一段 + 动手做出一个看得见的结果。

每周 2 讲的理由：中间至少隔一天，让大脑在睡眠里整理；但不超过 4 天，否则上次学的东西会凉透。

每讲 40 分钟的分配（固定节奏，形成习惯）：

时间	做什么
0–5 分	读「本节目标」，知道自己要得到什么
5–20 分	读概念部分（含比喻和项目里的真实例子）
20–35 分	**动手**：照着「动手任务」做，出错了先自己试
35–40 分	做「自测题」，对答案；不懂的记进「术语表」

重要：如果某讲你 40 分钟没做完，**不要硬撑。**停下来，下次接着做。讲义是按"讲"组织的，不是按"天"。

全套目录（24 讲）

模块 0 • 热身：先把它跑起来（2 讲） ✅ 已完成

讲	标题	你会得到
0-1 ✅	打开黑框：终端与命令行	能在终端里走目录、看文件、跑命令，不再怕那个黑窗口
0-2 ✅	装好你的工作台	能自己把项目跑起来（`npm install` → `npm run dev`），并知道每步在干什么

模块 1 • 程序到底是什么（3 讲） ✅ 已完成

讲	标题	你会得到
1-1 ✅	双击之后发生了什么	能讲清"图标 → 进程 → 窗口"这条链路；理解进程与内存
1-2 ✅	一切都是文件	理解文件、路径、编码；能解释"为什么中文路径会出问题"
1-3 ✅	代码是怎么变成软件的	理解"源代码 → 构建 → 打包"三级流水线；知道 Node 与 Electron 各是什么

模块 2 • 认清记账簿的骨架（4 讲） ✅ 已完成

讲	标题	你会得到
2-1 ✅	主角登场：主进程与渲染进程	理解为什么要有两个进程；能解释"为什么界面不能直接读数据库"
2-2 ✅	传菜口：preload 与 IPC	能读懂一条完整的"界面请求 → 主进程执行 → 返回结果"链路
2-3 ✅	冷库：数据库与迁移	理解表、字段、主键；能解释"为什么改数据库要写迁移"
2-4 ✅	装修队：React 与界面	理解组件、状态、渲染；能说出界面是怎么"根据数据画出来"的

模块 3 • 学会读代码（4 讲） ✅ 已完成

讲	标题	你会得到
3-1 ✅	读代码的地图与心法	掌握"从入口顺着追"的读法；不再从头读到尾
3-2 ✅	追一条数据流：记一笔账	完整跟完"点保存 → 写进硬盘"，能自己画出来
3-3 ✅	读第二遍：类型与契约	理解 TypeScript 在帮你什么；能看懂接口定义
3-4 ✅	动手：给列表加一个新列	第一次改真实代码，并看到界面变化

模块 4 • 学会改代码（4 讲） ✅ 已完成

讲	标题	你会得到
4-1 ✅	改代码的安全流程	掌握"确认基线 → 分析 → 备份 → 改 → 验证 → 收尾"六步；知道怎么不弄坏数据
4-2 ✅	从需求到设计（上）	拿到需求先问三个问题；判断需求类型；列出改动清单
4-3 ✅	从需求到设计（下）	跨三层写出真实功能（排序），并通过验收清单
4-4 ✅	出错怎么办：读懂报错	掌握五种常见报错与排查顺序；不再一报错就慌

模块 5 • 质量与安全（3 讲） ✅ 已完成

讲	标题	你会得到
5-1 ✅	那些 verify 脚本在防什么	明白自动化验证的价值；能按改动范围选择该跑哪些检查
5-2 ✅	数据安全：备份、迁移、加密	理解软删除、自动备份、应用锁的设计取舍与边界
5-3 ✅	隐私与离线：这个项目的结构性优势	能自己验证"没有联网代码"，并能讲清隐私优势

模块 6 • 打包与发布（2 讲） ✅ 已完成

讲	标题	你会得到
6-1 ✅	从源码到安装包	理解打包做了什么；能自己出一个安装包并验证
6-2 ✅	发布：网站、下载、被搜到	能把软件交给陌生人使用；理解签名与信任的成本

模块 7 • 独立造东西（2 讲） ✅ 已完成

讲	标题	你会得到
7-1 ✅	从零设计一个小软件	独立完成需求拆解与数据模型设计
7-2 ✅	把它做出来并交付	完整的开发计划、工作节奏与交付自查清单

全套 24 讲已完成 ✅

讲义文件在 `course\pdf\` 目录，共 25 份 PDF（总纲 1 份 + 24 讲）。

生成命令：`npm run course` ｜ 自检命令：`npm run verify:course`

每讲的固定结构

每讲 PDF 都长这样，你熟悉之后可以直接跳到想看的部分：

1. **本节目标** —— 读完能做到什么（可验证的句子，不是"了解"）
2. **概念** —— 用现实比喻建立直觉，再引出术语
3. **在项目里看真实文件** —— 指出具体路径，让你亲眼确认
4. **动手任务** —— 明确步骤 + 预期结果 + 卡住了怎么办
5. **自测题** —— 5 道题，附答案与解析
6. **术语表** —— 本讲出现的新词，一句话解释

学习建议（认真读这一段）

1. 允许自己不懂。 讲义里会故意留下"这里先记住，模块 4 会解释"的钩子。纠缠每个细节会让你在第 2 讲就停下。

2. 动手时出错是常态，不是失败。 报错是计算机在跟你说话，只是它说话很难听。模块 4-4 专门教你怎么听懂它。

3. 不要在真数据上做实验。 讲义里的动手任务都设计在临时目录里。**你的账本数据（`D:\加速器`）永远不要拿来试手。**

4. 卡住超过 15 分钟就问。 这不是学校考试，卡住不代表你不行，往往只差一句话。

5. 学完模块 4 你就已经能改这个软件了。 模块 5–7 是把你从"改"提升到"造"。

这套讲义随项目更新。每一讲末尾会标注它与当前代码版本的对应关系。